Counterparty Inscriptions

aka **Bitcoin Counters** — on-chain files stored in Bitcoin witness data, attached to Counterparty assets.

Build reference · working draft · June 2026

Naming. The protocol is **Counterparty Inscriptions** (branded **Bitcoin Counters**, casually **Counters**); a single item is a **Bitcoin counter**.

1. What a Bitcoin counter is

A Bitcoin counter is a **numbered inscription** — a file committed permanently to Bitcoin, stored in **witness data**, and tied to a **Counterparty asset** minted in the same transaction. The number marks its place in the sequence, counting up from zero; the file is its content; the Counterparty asset is the handle by which it is identified, owned, and transferred.

Once minted, the asset behaves like any Counterparty asset — it can be held, sent, and traded under Counterparty's rules — and the file stays fixed on-chain as the permanent content bound to the asset's creation. There is no separate token for the file: the asset is the unit of ownership, the number is its place in the sequence, and the file rides with its mint.

The attached asset can be **any** Counterparty asset — named, numeric, or subasset — so a Bitcoin counter inherits Counterparty's full naming system for free.

How it compares

Counterparty Inscriptions is its own protocol; here is how it compares to Ordinals and Bitcoin Stamps. It stores file data in witness like Ordinals, and takes its ownership from a Counterparty asset like Stamps.

	Ordinals	Bitcoin Stamps	Bitcoin Counters
File stored in	witness data	bare multisig (UTXO set)	witness data
Ownership / identity	a satoshi, tracked off-protocol	a Counterparty asset	a Counterparty asset
Transfer	move the satoshi	Counterparty send	Counterparty send

The comparison is orientation only — the definition stays “files in witness data attached to Counterparty assets.”

2. The three building blocks

Witness data storage

The file is written into the witness of a Bitcoin transaction using a data envelope: an `OP_FALSE OP_IF … OP_ENDIF` wrapper that holds arbitrary bytes as an unexecuted no-op. This is the same generic envelope structure Ordinals uses, so existing envelope-parsing code is reusable — the envelope is plain Bitcoin script, not anyone's property. Witness bytes receive the witness discount (~4× cheaper by weight than output data), making on-chain file storage relatively economical, and they live outside the UTXO set (unlike Bitcoin Stamps, which store their data in the UTXO set via bare multisig). What marks these particular bytes as a Bitcoin counter is the first value pushed inside the envelope — the marker — not the envelope itself.

Commit / reveal

Writing to the witness takes two transactions. A taproot script-path spend can only spend an output that **already commits** to the script being revealed, and an input cannot reference an output created in the same transaction. So a commit transaction first creates the committing output, and a reveal transaction then spends it, exposing the file in the witness. This is simply how data is written to the witness; there is no single-transaction alternative.

Counterparty assets

Every Counterparty asset maps to a canonical 64-bit integer `asset_id`. Ownership is a ledger balance attached to a Bitcoin address inside Counterparty's accounting — not coins that ride on outputs. The asset is what gets owned and transferred, and Counterparty consensus is the source of truth for who holds it.

3. Counterparty asset model

Type	`asset_id` range	Display	Issuance cost

BTC / XCP	`0` / `1`	reserved	n/a (cannot back a Bitcoin counter)
Named	base-26 of a 4–12 char A–Z name not starting with `A`; below $26^{12}+1$	decode `asset_id` (e.g. `RAREPEPE`)	0.5 XCP burned
Numeric (unnamed)	$26^{12}+1$ … $2^{64}-1$	`A` + integer	free (BTC fee only)
Subasset	the backing numeric id	`PARENT.CHILD` (from the issuance's compacted longname)	parent-owner only

> **Critical constraint — one asset per transaction.** A Bitcoin transaction carries a single Counterparty message, and an issuance message mints exactly one asset. There is no multi-issuance message type. The only batching primitive Counterparty has (MPMA) batches *sends* of existing assets, never issuances. This single fact is what lets a Bitcoin counter omit any asset-reference field: "the asset minted in this transaction" always resolves to exactly one asset.

4. The Bitcoin counter envelope (witness format)

The file is serialized as data pushes inside the envelope, revealed in the witness of a taproot script-path spend. The leaf is made spendable by a key check (`<32-byte pubkey> OP_CHECKSIG`); since the envelope is a skipped no-op, the key check may sit before or after it (ord places it first), so a parser must **scan** the script for the envelope rather than assume a fixed position.

```
OP_FALSE
OP_IF
  OP_PUSH "COUNT"           # protocol marker (freeze before launch)
  OP_PUSH 0x01              # tag 1 (content type): a 1-byte data push (OP_PUSHBYTES_1 0x01)
  OP_PUSH <content_type>     # MIME type of the body, e.g. image/png
  OP_0                      # empty separator (0x00): fields end, body begins
  OP_PUSH <body chunk 1>     # file bytes, ≤ 520 bytes per push
  OP_PUSH <body chunk 2>
  ...
OP_ENDIF
<32-byte x-only pubkey>
OP_CHECKSIG
```

Tag	Name	Encoding	Required	Meaning
—	marker	data push of `COUNT`	yes	First push (`OP_PUSHBYTES_5 434f554e54`). Identifies a Bitcoin counter.
1	content_type	data push of `0x01`	yes	A 1-byte push of `0x01` (`OP_PUSHBYTES_1 0x01`) — ord's standard encoding; the *following* push is the MIME type, e.g. `image/png`. The `OP_1` (`0x51`) form also appears on-chain — parse both.
—	separator	`OP_0` (`0x00`)	yes	Empty push. Ends the fields, starts the body.

The body is the file, concatenated from all pushes after the empty separator. Individual pushes are capped at 520 bytes (a taproot rule); large files span multiple pushes. There is **no asset-reference field** — the one-issuance-per-tx guarantee makes it unnecessary.

Tags follow the "it's okay to be odd" rule for forward compatibility: an unrecognized *even* tag must invalidate the counter; an unrecognized *odd* tag may be ignored.

> **Byte-for-byte, this is ord's envelope with a different marker.** The wrapper, the content-type tag, the `OP_0` separator, and the ≤520-byte body chunks match Ordinals — only the marker differs (`COUNT` vs `ord`) — so ord's envelope parser is reusable. One subtlety to fix: tag 1 is a **1-byte data push of `0x01`**, not `OP_1`. (ord's docs once showed `OP_1`, which actually broke parsing; inscriptions using the `OP_PUSHNUM_1` form went unrecognized, then were treated as cursed before the jubilee at block 824,544 and blessed after.) Both forms now exist on-chain, so COUNT should **create with the `0x01` push but accept either when parsing** — a rule the test vectors must fix.

5. Transaction anatomy

A Bitcoin counter is created with commit/reveal, where the reveal is also the Counterparty issuance.

```
Commit transaction
  → output: P2TR address committing to the tapscript that holds the "COUNT" envelope
     (no Counterparty data at all; just parks BTC at an address that commits to the file)

Reveal transaction  (= the mint)
  inputs:
    vin[0]  ← a funded wallet UTXO (holds ≥ 0.5 XCP for a named asset)
    vin[k]  ← spends the committed P2TR output → its witness reveals the file
  outputs:
    vout[0] ← OP_RETURN carrying the Counterparty issuance (mints the asset)
    vout[1] ← asset recipient / change
```

A Bitcoin counter's identity comes from the asset, not from any particular input, so the envelope may be revealed in **any** input — you are free to order inputs so a funded wallet UTXO is first. The Counterparty issuance must use a normal `OP_RETURN` (or bare multisig), **not** Counterparty's own taproot-envelope mode, so the issuance message and the `"COUNT"` envelope never collide in the witness (see §11). The issuance description may be empty — the file lives in the witness.

6. Binding

The binding is established by co-location in one transaction: the Bitcoin counter is the file in the `"COUNT"` envelope; the asset is whatever the Counterparty issuance in the same transaction mints. That asset is the counter's identity. No reference field is needed, because Counterparty mints at most one asset per transaction.

7. Who issues it, and how the XCP works

Two separate things flow through the mint, and they never touch:

- **BTC path (real coins).** Your wallet funds the commit's P2TR output; the reveal spends it. This pays for the data reveal and fees. It is the only thing that flows commit → reveal.
- **XCP path (a ledger entry).** XCP is a balance attached to your address in Counterparty's ledger. It is never “sent.” When the issuance is processed, Counterparty debits 0.5 XCP from the source address's balance (the “burn,” named assets only) and credits the new asset to it.

> **The source rule:** Counterparty reads the issuer/source from the transaction's **first input**. counter places the funded wallet UTXO that holds the XCP at `vin[0]`, so that one address is the source, the address the 0.5 XCP burn is debited from, and the default owner the asset is credited to — all at once. The committed P2TR output is just another input (`vin[k]`) that reveals the file; it is never the source and never needs to hold XCP. The only requirement is that the `vin[0]` address be a type Counterparty accepts as a source — which, post-v11, includes taproot (`bc1p`), so the taproot-only wallet qualifies.

8. Identity & numbering

A Bitcoin counter has two identifiers, either of which references it uniquely:

- **The Counterparty asset** — its `asset_id` (and, for display, the asset name or subasset longname), read from the issuance.
- **The counter number** — a global sequential index assigned to each *valid* counter in mint order: by block height, then by transaction position within the block. Numbers start at **0** and count up with **no gaps** — the same scheme Ordinals uses for inscription numbers. Since a transaction holds at most one `"COUNT"` envelope, ordering is fully determined. Invalid attempts (failed issuance, malformed envelope, re-issuance of an existing asset) are skipped and do not advance the counter. Numbering is computed over the canonical chain, so a reorg can renumber recent mints until they are buried.

9. Ownership & transfer

Whoever holds the Counterparty asset balance owns the Bitcoin counter. Transfer happens via normal Counterparty sends. The witness file never moves — it is permanent provenance pinned to the mint transaction. Only the asset changes hands.

10. Validity rules

A transaction is a valid Bitcoin counter if and only if **all** hold:

1. The reveal's witness contains a well-formed `"COUNT"` envelope (marker, tag-1 content_type, empty separator, body).
2. The same transaction contains a Counterparty issuance that **successfully mints an asset** under Counterparty consensus. A rejected issuance yields no counter, even though the file bytes are permanently on-chain.
3. That issuance is the asset's **first** issuance (the mint). Later re-issuances or description edits never replace the content.
4. Exactly **one** `"COUNT"` envelope is present.
5. The minted asset is not BTC (`0`) or XCP (`1`).

Because validity depends on rule 2, the indexer cannot be a pure witness scanner — it must confirm issuance success against Counterparty consensus. An asset can even appear in a block explorer's index yet still be an invalid issuance, so the indexer trusts Counterparty's verdict, not an explorer's listing.

11. Why the file goes in your own envelope, not the Counterparty message

Counterparty has a **native** mode for embedding a file directly in an issuance (API: `encoding=taproot`, `inscription=true`, `mime_type`) — in fact this mode produces an actual Ordinals inscription, since Counterparty offers an Ordinals-compatible envelope. In that mode the file **and its MIME type are part of the Counterparty message itself**, so Counterparty parses and validates them — and can reject the whole issuance. As a real counter-example, the native mint `SILKROADSONG` (itself an Ordinals inscription, not a counter) was rejected outright with `invalid: Invalid mime type: audio/ogg;codecs=opus` — a file a counter would carry with no objection.

Counterparty Inscriptions deliberately keeps the file **out** of the Counterparty message:

- the file and MIME live in your own `"COUNT"` witness envelope, which Counterparty never parses;
- the Counterparty side is a plain `OP_RETURN` issuance with an empty description;
- so Counterparty has no file or MIME to validate — a mint can **never** be rejected for an unsupported file type.

The trade-off, stated plainly: you gain total freedom over file types (opus audio, anything), but you lose Counterparty's media-level validity gating entirely. Your indexer is the **sole** authority on whether a file is acceptable.

> **Why the two encodings don't collide:** Counterparty parses a transaction by first checking the `OP_RETURN`. If the `OP_RETURN` pushdata is the literal `CNTRPRTY`, it treats the tx as a taproot commit and reads the message from the witness. Otherwise it RC4-decodes the `OP_RETURN` as a legacy message and never looks at the witness. So putting a legacy `OP_RETURN` issuance in the reveal keeps Counterparty out of your `"COUNT"` witness.

12. Indexer design

The indexer is essentially a **join**: “transactions with a valid `"COUNT"` envelope” on one side, “successful first issuances” on the other, matched by txid, then numbered.

> **Architectural rule:** do not reimplement Counterparty consensus. Treat Counterparty Core as an oracle (run it or hit its API) and consume its parsed issuances and valid/invalid determination per block. Asset naming, XCP balances, issuance fees, and the validity decision are nuanced (the MIME rejection above shows how much). A second implementation will eventually disagree with the real one. Your code owns only witness parsing, the join, and numbering.

Pipeline:

1. Pull each block from a Bitcoin node (RPC or an indexed source).
2. For every transaction, scan witnesses for a `"COUNT"` envelope (a witness envelope parser keyed on the `COUNT` marker).
3. For each tx with exactly one envelope, ask Counterparty whether the same tx has a successful issuance; get its `asset_id`, whether it is the first issuance, and validity.
4. Apply the validity rules, assign the next counter number, store the record.

```
for each tx in block order:
    env = parse_count_envelope(tx.witnesses)    # OP_FALSE OP_IF "COUNT" ... OP_ENDIF
    if env is None: continue
    if count_envelopes_in(tx) != 1: continue     # rule 4

    iss = counterparty_issuance_in(tx)           # Counterparty allows at most one
    if iss is None: continue                      # rules 1 + 2
```

```
    if not counterparty_mint_succeeded(iss): continue   # rule 2
    if not is_first_issuance(iss.asset_id): continue    # rule 3
    if iss.asset_id in (0, 1): continue                 # rule 5

    record_counter(
        asset_id     = iss.asset_id,             # from the issuance, not the envelope
        name         = resolve_name(iss),        # named / "A"+num / subasset longname
        content_type = env.content_type,
        content      = env.body,
        mint_txid    = tx.id,
        number       = next_counter_number(),  # global, starts at 0, valid only
    )
```

Source/issuer is not the indexer's to derive: Counterparty resolves it from the first input, and the indexer simply reads it off the issuance.

13. Things to freeze before launch

- **Marker and tag encoding.** The literal `"COUNT"` marker, and tag `1` = content_type as a `0x01` data push — plus whether to also accept the `OP_1` (pushnum) form when parsing. (Optionally register a metaprotocol identifier string.)
- **Genesis (no activation height).** Counter #0 is the first valid counter detected on-chain — the genesis is data-defined, not a fixed block. The indexer's scan floor is taproot activation (block 709,632, the earliest a witness envelope can exist), which is an implementation detail, not a protocol constant.
- **Reorg handling.** Numbers are a pure function of canonical-chain order, so on a reorg the indexer simply recomputes the affected blocks — no rollback machinery. Treat the most recent numbers as non-final until a few confirmations.
- **Issuance mapping.** Exactly which Counterparty API field/endpoint corresponds to “successful first issuance” (the issuance that creates the asset, with valid status).
- **Asset-type policy.** Accept all asset types, or restrict to a subset. Default: accept all.
- **Supply semantics.** Whether to require `lock` and `divisible=0` for 1-of-1 behavior, or allow fungible / divisible counters.
- **Source rule (resolved).** The source is `vin[0]`; counter puts the XCP-holding wallet UTXO there, so the issuer, burn address, and default owner are that one input. No special derivation — just the standard Counterparty first-input rule.

Status: working draft / build reference. Everything hard (asset identity, ownership, issuance validity, the XCP burn) lives in Counterparty; the Counterparty Inscriptions layer is a witness envelope plus an indexing convention. Treat Counterparty Core as the source of truth and keep this layer thin.